

# Práctica 4: Paralelización mediante OpenMP y programación asíncrona del análisis forense de manipulación de imágenes digitales

Joaquín Sigüenza Chilar and Linxi Jiang

Computación de Alto Rendimiento  
Grado en Ingeniería en Inteligencia Artificial  
Universidad de Alicante

**Resumen** En esta memoria paralelizamos un sistema de análisis forense de manipulación de imágenes digitales con OpenMP y programación asíncrona en C++. Partiendo del código secuencial que se nos ha proporcionado, hemos aplicado dos niveles de paralelismo: paralelismo de datos con OpenMP en los bucles internos de la clase `Image<T>` y de la DCT por bloques, y paralelismo funcional con `std::async` sobre los cinco procesos independientes (SRM 3x3, SRM 5x5, ELA, DCT directa y DCT inversa). Las pruebas se han realizado en una máquina con un AMD Ryzen 7 7435HS (8 núcleos físicos, 16 hilos lógicos), midiendo tiempos, speed-up y eficiencia con 1, 2, 4, 8 y 16 hilos. La versión OpenMP llega a un speed-up de  $5,39\times$  con 16 hilos. La versión combinada se queda en  $5,01\times$  en esa misma configuración, pero gana a la OpenMP pura en todas las demás: con 2 hilos OpenMP por proceso es un 24,7% más rápida. La configuración que mejor nos ha funcionado es la combinada con 8 hilos OpenMP, que da un speed-up de  $4,78\times$  con una eficiencia de 0,60. En definitiva, los dos modelos de paralelismo se complementan y combinarlos resulta beneficioso siempre que se controle el oversubscription.

**Keywords:** OpenMP · `std::async` · paralelismo · análisis forense · SRM · ELA · DCT.

## Repositorio del código:

[https://github.com/Ximsig/Practica\\_4](https://github.com/Ximsig/Practica_4)

## Introducción y objetivos

El objetivo de la práctica es aprender a paralelizar una aplicación en una máquina paralela de memoria centralizada mediante hilos, con el estilo de variables compartidas. Para ello hemos estudiado la API de OpenMP, hemos aplicado distintas estrategias de paralelismo y también hemos usado la API de C++ de programación asíncrona (`std::async`) para explotar el paralelismo funcional. Sobre todo ello aplicamos métodos para estimar las ganancias máximas y la eficiencia de la paralelización, todo en un problema de suficiente complejidad: el análisis forense de manipulación de imágenes digitales.

## Tarea 0.1: Entrenamiento previo OpenMP

### ¿Para qué sirve la variable `chunk`?

`chunk` es el tamaño del bloque de iteraciones que se le asigna a cada hilo en el reparto del bucle. Como se usa dentro de `schedule(dynamic, chunk)`, lo que hace es lo siguiente: cada hilo coge `chunk` iteraciones consecutivas, las procesa, y cuando acaba pide otro bloque del mismo tamaño al planificador hasta que se acaben las iteraciones. En el ejemplo, con  $N = 1000$  y `CHUNKSIZE = 100` tenemos 10 bloques de 100 iteraciones que se reparten bajo demanda entre los hilos.

El tamaño del `chunk` es un compromiso. Un `chunk` pequeño favorece el balanceo de carga (los hilos rápidos siguen cogiendo trabajo mientras los lentos terminan), pero introduce más sobrecarga de planificación al pedir bloques más a menudo. Un `chunk` grande reduce esa sobrecarga, pero penaliza el balanceo si las iteraciones tienen costes muy distintos: un hilo se puede quedar bloqueado con un bloque pesado mientras los demás ya han acabado y permanecen inactivos.

### Explicación del `pragma`

```
#pragma omp parallel shared(a,b,c,chunk) private(i)
```

`#pragma omp parallel` abre una región paralela: el hilo maestro crea un equipo de hilos (por defecto, tantos como núcleos lógicos haya, o lo que diga `OMP_NUM_THREADS`) y todos ejecutan el bloque que viene a continuación. Las cláusulas `shared` y `private` controlan el ámbito de las variables dentro de la región, es decir, si son únicas y compartidas o si cada hilo tiene su propia copia local.

**Uso de `shared(a,b,c,chunk)`.** Las cuatro variables son únicas en memoria y todos los hilos acceden a la misma dirección. La razón de declararlas `shared` se ve mirando para qué sirve cada una. Los vectores `a` y `b` se leen en paralelo durante el cálculo, y al ser solo lecturas no hay condiciones de carrera, así que es seguro compartirlas. `c` sí se escribe en paralelo, pero cada hilo escribe en una posición distinta (`c[i]`, con `i` privada), así que tampoco hay conflicto. `chunk` solo se consulta para configurar el `schedule`, así que no necesita protección.

Si las declaráramos como `private`, cada hilo tendría su propia copia local sin inicializar y los resultados escritos en `c` no quedarían disponibles fuera de la región paralela, con lo que el programa no haría lo que queremos.

**Uso de `private(i)`.** La variable de inducción del bucle tiene que ser privada porque cada hilo recorre su propio rango de iteraciones. Si `i` fuera compartida, los hilos se pisarían el contador y tendríamos una condición de carrera evidente. OpenMP convierte automáticamente en privada la variable de inducción de un `omp for`, pero declararlo explícitamente es útil y deja claro al lector el comportamiento de esa variable.

### ¿Para qué sirve `schedule`? Otras posibilidades

`schedule` define cómo se reparten las iteraciones entre los hilos, y elegir bien la política tiene impacto directo en el rendimiento dependiendo de cómo sea el problema.

`static[, chunk]` calcula el reparto al inicio de forma cíclica, asignando bloques de tamaño `chunk` (o  $N/\text{num\_threads}$  si no se pone). Tiene sobrecarga mínima y resulta apropiado cuando todas las iteraciones cuestan lo mismo.

`dynamic[, chunk]` hace que los hilos pidan bloques bajo demanda según van terminando. Mete más sobrecarga, pero el balanceo es muy bueno cuando las iteraciones tienen costes dispares.

`guided[, chunk]` es parecido a `dynamic`, pero el tamaño del bloque va decreciendo exponencialmente hasta llegar a `chunk`. Es un punto medio entre las dos anteriores.

`auto` le deja la decisión al compilador o al runtime, y `runtime` permite escoger la política en tiempo de ejecución a través de `OMP_SCHEDULE`, lo cual resulta útil para hacer pruebas sin tener que recompilar cada vez.

### Medidas de rendimiento en OpenMP

Para medir tiempos en las secciones paralelizadas con OpenMP usamos `omp_get_wtime()`, que devuelve segundos en un `double`. A partir de ese tiempo calculamos dos indicadores: el speed-up,  $S(p) = T_{\text{secuencial}}/T_{\text{paralelo}}(p)$ , donde  $p$  es el número de hilos; y la eficiencia,  $E(p) = S(p)/p$ , que mide el aprovechamiento real de los núcleos y cuyo valor ideal es 1,0.

OpenMP también ofrece funciones de introspección útiles, como `omp_get_num_threads()` para saber cuántos hilos hay activos y `omp_get_thread_num()` para conocer el ID del hilo actual.

A nivel teórico tenemos la Ley de Amdahl, que pone una cota al speed-up máximo en función de la fracción serie del programa:

$$S_{\max}(p) = \frac{1}{f + \frac{1-f}{p}}, \quad (1)$$

donde  $f$  es la fracción no paralelizable. Por mucho que se incremente el número de hilos, la parte serie del programa impone un techo. Para un análisis más detallado existen herramientas externas como `perf`, `gprof`, Intel VTune o `likwid`, que permiten ver fallos de caché, mispredicciones de salto y otros eventos hardware que ayudan a entender el comportamiento del programa paralelo.

### Tarea 0.2: Entrenamiento previo `std::async`

#### ¿Para qué sirve el parámetro `std::launch::async`?

`std::launch::async` es una política de lanzamiento que obliga al runtime de C++ a ejecutar la función en un hilo nuevo, de forma totalmente asíncrona

e independiente del hilo principal. Si no la indicamos (o dejamos el valor por defecto), el compilador podría decidir no crear un hilo nuevo si los recursos del sistema están saturados, ejecutando la tarea de forma diferida. Al poner `std::launch::async` explícito, garantizamos paralelismo real desde el momento de la llamada.

### Comparativa de tiempos: `std::launch::async` frente a `std::launch::deferred`

Al ejecutar con `std::launch::async`, el tiempo total ronda los **3000 ms**. Las dos tareas se lanzan a la vez en hilos separados; la tarea 1 (2000 ms) y la tarea 2 (3000 ms) se solapan en el tiempo, así que el total es el de la tarea más larga. Confirmado que se ejecutan en paralelo.

Si modificamos el código para usar `std::launch::deferred`, el tiempo sube a unos **5000 ms**. La diferencia es que `deferred` aplica evaluación perezosa (*lazy evaluation*): no se crea ningún hilo nuevo y la tarea se pospone hasta que el hilo principal llama a `task1.wait()` o `task2.get()`. Se ejecutan secuencialmente en el mismo hilo que hace la llamada (2000 ms + 3000 ms).

### Diferencia entre `wait` y `get` en `std::future`

Los dos métodos bloquean al hilo que los invoca hasta que la tarea asíncrona termine, pero no son lo mismo:

- **`wait()`**: Solo espera a que el estado compartido del `std::future` esté listo. No devuelve nada (retorna `void`) y no altera el estado del objeto, así que se puede llamar varias veces. Útil cuando la tarea se ejecuta por sus efectos secundarios (escribir en un fichero, modificar una variable pasada por referencia) o simplemente para sincronizar hilos.
- **`get()`**: Espera a que la tarea termine *y* además extrae y devuelve el valor de la función (en el ejemplo, el `id` de tipo `int`). Lo importante: `get()` mueve el estado interno del `future`, dejándolo en estado inválido tras llamarlo. Por eso `get()` **solo se puede llamar una vez** por cada `std::future`.

### Ventajas de `std::async` frente a `std::thread`

`std::async` es una abstracción de más alto nivel que la gestión manual con `std::thread`, y aporta tres ventajas en C++ moderno:

1. **Gestión simplificada de retornos**: `std::async` devuelve automáticamente un `std::future`, que hace trivial recuperar el valor de retorno de una función ejecutada en otro hilo. Con `std::thread`, recuperar un valor es bastante más engorroso: variables compartidas con *mutex*, punteros pasados por referencia o `std::promise` explícito.

2. **Propagación de excepciones:** Si la función lanzada con `std::async` lanza una excepción, el estado compartido la captura y la relanza en el hilo principal cuando llamamos a `get()`. Con `std::thread`, una excepción no capturada dentro del hilo termina el programa abruptamente con `std::terminate`.
3. **Gestión automática de recursos:** Al delegar el control en el runtime de la librería estándar, `std::async` se encarga del ciclo de vida del hilo (no hay que acordarse de llamar a `join()`). Además, la implementación puede usar un *thread pool* en lugar de instanciar un hilo del SO por cada llamada, lo que reduce sobrecarga.

### Tarea 0.3: Entrenamiento previo `std::vector`

#### ¿Cuál de las dos formas de inicializar es más eficiente?

La segunda — inicialización con tamaño (`std::vector<float>v2(N)`) y acceso directo (`v2[i] = i`) — es bastante más eficiente. La razón está en cómo gestiona la memoria internamente `std::vector`.

En el primer caso, `v1` se construye con capacidad cero. Cada `push_back` mira si hay sitio y, cuando se llena el buffer, realoca uno nuevo (típicamente del doble de tamaño), copia o mueve todos los elementos al bloque nuevo y libera el anterior. Esa realocación se repite del orden de  $\log_2(N) \approx 24$  veces para  $N = 10^7$ . Aunque amortizadamente `push_back` sea  $O(1)$ , en la práctica hay muchas llamadas a `new` y `delete` y un buen montón de copias innecesarias. Además `push_back` incrementa `size` y comprueba la capacidad en cada iteración, lo que añade un coste fijo.

En el segundo caso solo se hace una reserva de memoria al construir el vector. El bucle queda como escrituras directas sobre un buffer ya existente, sin saltos al heap ni copias, y `operator[]` no comprueba nada. El coste es  $O(N)$  limpio, sin sobrecargas.

Como apunte: si la lógica del programa exige usar `push_back` pero conocemos el tamaño aproximado, una llamada previa a `v.reserve(N)` evita las realocaciones manteniendo `size()` en cero. Es un truco sencillo que combina lo mejor de los dos enfoques.

#### ¿Podría ocurrir algún problema al paralelizar los dos bucles?

La respuesta es distinta para cada bucle, y la diferencia es importante.

El primer bucle, el del `push_back`, sería un problema serio porque `std::vector::push_back` no es *thread-safe*. Si dos hilos lo llaman a la vez, hay condición de carrera sobre el campo `size`, sobre el puntero interno y, sobre todo, sobre la posible realocación. Esto puede dejar elementos perdidos o duplicados, doble liberación de memoria, corrupción del heap y fallos de segmentación. Para paralelizarlo de forma segura habría que serializar el acceso con `#pragma omp critical` o un `mutex`, pero entonces se pierde casi toda la ganancia que buscábamos paralelizando.

El segundo bucle, el de `v2[i] = i`, no tiene ningún problema y se paraleliza limpiamente. Cada iteración escribe en una posición distinta del vector ya reservado, no hay condiciones de carrera ni dependencias entre iteraciones, y basta con un `#pragma omp parallel for`. Es el patrón *data-parallel* clásico.

La conclusión práctica: cuando paralelizamos bucles que rellenan contenedores, lo recomendable es redimensionar primero y usar acceso por índice. El código que se nos ha dado para la Tarea 1.3 ya sigue este patrón: los buffers internos de `Image<T>` se reservan en el constructor con `new T[height*width*channels]`, lo que facilita la paralelización con OpenMP en las operaciones sobre imágenes.

## Tarea 1.1: Análisis del código e identificación de procesos

### Procesos identificados

El programa principal (`main.cc`) carga una imagen y le aplica cinco procesos de análisis forense, cada uno de los cuales genera un PNG de salida. Los cinco procesos son independientes entre sí: todos parten de la misma imagen original, pero ninguno necesita el resultado de los demás.

El primero es la convolución con el kernel SRM de tamaño  $3 \times 3$  con `compute_srm(image, 3)`, que produce `srm_kernel_3x3.png`. El segundo es lo mismo pero con el kernel SRM de  $5 \times 5$ , generando `srm_kernel_5x5.png`. El tercero es el Error Level Analysis (ELA) con calidad JPEG 90, implementado en `compute_ela`: recomprime la imagen, la resta de la original para evidenciar zonas con distinto nivel de compresión, y guarda el resultado como `ela.png`. El cuarto y el quinto son las dos variantes de la Transformada Discreta del Coseno por bloques de  $8 \times 8$ : la versión inversa, que anula los coeficientes de bajas frecuencias y reconstruye la imagen para resaltar las zonas con menor energía en altas frecuencias (`dct_invert.png`); y la versión directa, que muestra los coeficientes DCT en bruto (`dct_direct.png`).

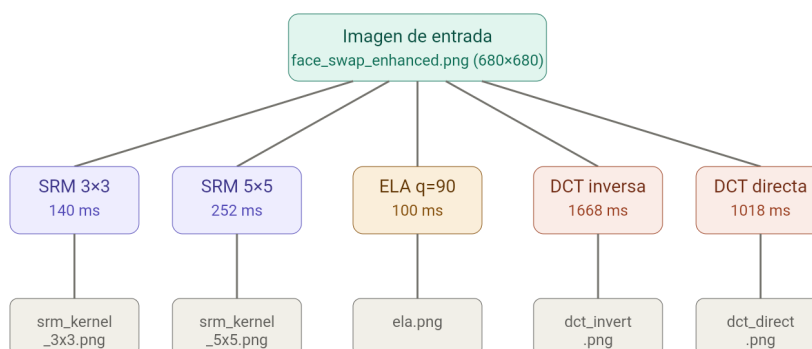
### Diagrama de dependencias

El diagrama de dependencias es muy simple: un único nodo de entrada (la imagen original) se ramifica en cinco caminos paralelos, cada uno con su proceso y su salida. No hay ninguna arista entre los cinco procesos. Esta estructura es ideal para paralelismo funcional con `std::async`: cada proceso se puede ejecutar en un hilo distinto sin coordinación. Por dentro, cada proceso tiene bucles densos sobre los píxeles que son candidatos a paralelismo de datos con OpenMP. La Figura 1 ilustra todo esto.

## Tarea 1.2: Análisis de cada proceso y tiempos

### Diagramas de flujo

Aunque hay cinco procesos, son tres tipos distintos: SRM (que cubre tanto la versión  $3 \times 3$  como la  $5 \times 5$ ), ELA y DCT (que cubre tanto la directa como



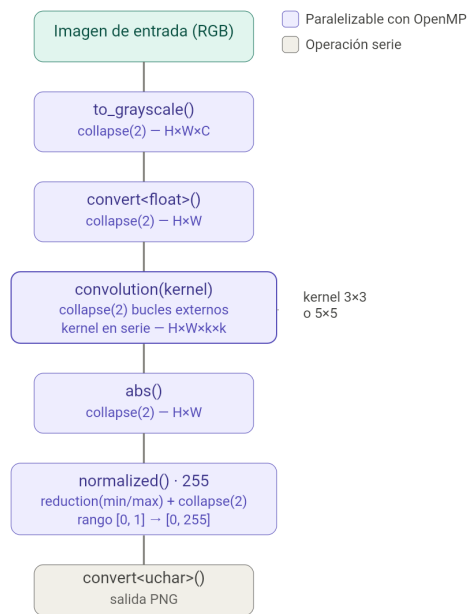
**Figura 1.** Diagrama de dependencias entre procesos. La imagen de entrada se ramifica en cinco procesos independientes que pueden ejecutarse en paralelo, generando cada uno su propio fichero PNG de salida.

la inversa). Vamos a ver los pasos internos de cada uno e identificar los puntos paralelizables. Las Figuras 2, 3 y 4 resumen cada flujo.

**Proceso SRM.** Coge la imagen original y la convierte a escala de grises con `to_grayscale()`, después la pasa a coma flotante con `convert<float>()`. Aplica `convolution(kernel)` con el kernel SRM correspondiente, calcula el valor absoluto con `abs()`, normaliza el resultado al rango  $[0, 1]$  con `normalized()`, lo escala por 255 y finalmente lo convierte a `unsigned char`. La operación dominante es la convolución, un bucle cuádruple de tamaño  $height \times width \times k \times k$  sobre toda la imagen. Es el principal candidato a paralelismo de datos con OpenMP. El resto son bucles triples  $height \times width \times channels$ , también paralelizables aunque pesan menos.

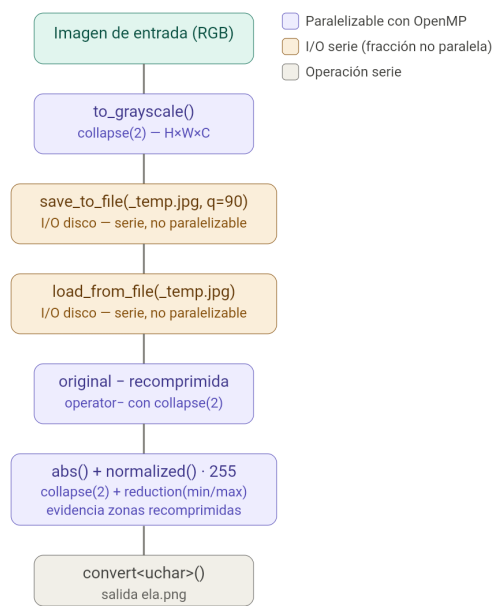
**Proceso ELA.** Empieza convirtiendo la imagen a escala de grises, la guarda como JPEG con calidad 90 (`save_to_file`), la vuelve a leer (`load_from_file`) y la resta a la original. El resultado pasa por `abs()`, `normalized()` y una multiplicación por 255 antes de convertirlo a `unsigned char`. Las operaciones aritméticas se aprovechan indirectamente de OpenMP a través de los operadores de `Image` ya paralelizados, pero la parte de I/O (escribir y leer el JPEG temporal) es estrictamente serie y supone una fracción no paralelizable del proceso.

**Proceso DCT.** Convierte la imagen a flotante y a escala de grises, la divide en bloques de  $8 \times 8$  con `get_blocks(8)` y, para cada bloque, calcula la DCT directa con `dct::direct` (un bucle cuádruple sobre el bloque). Si la variante es la inversa, anula los coeficientes de bajas frecuencias y reconstruye el bloque con `dct::inverse`; si no, asigna directamente los coeficientes al bloque. Al final



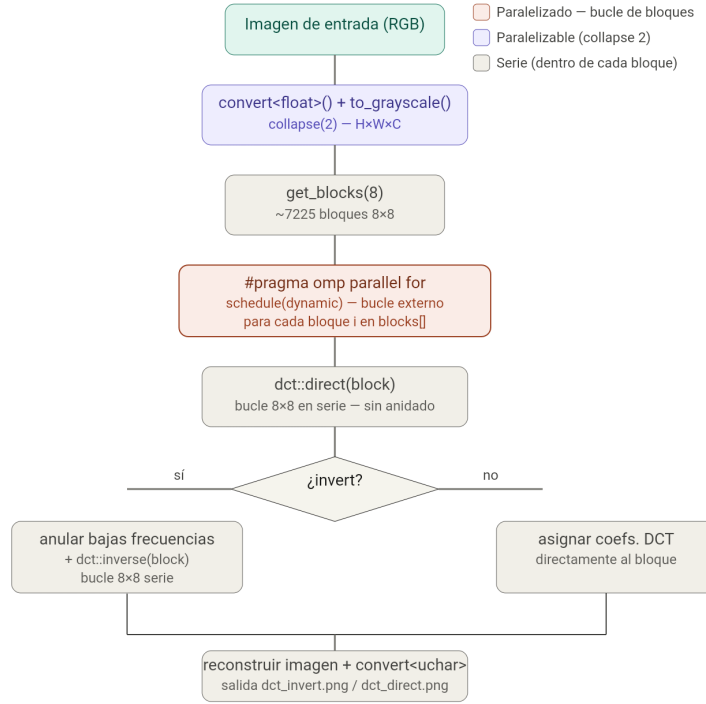
**Figura 2.** Diagrama de flujo del proceso SRM. Todas las operaciones son paralelizables con OpenMP; la convolución con el kernel es el bucle más caro y se paraleliza con `collapse(2)` sobre los dos bucles externos.





**Figura 3.** Diagrama de flujo del proceso ELA. Las operaciones de I/O sobre el JPEG temporal (en naranja) son estrictamente serie y limitan el speed-up alcanzable por este proceso.

convierte la imagen otra vez a `unsigned char`. El bucle externo sobre bloques es el candidato natural a OpenMP: para una imagen de  $680 \times 680$  píxeles tenemos 7 225 bloques ( $85 \times 85$ ), cada uno independiente del resto. Granularidad gruesa, ideal.



**Figura 4.** Diagrama de flujo del proceso DCT. El bucle externo sobre bloques es lo que se paraleliza con OpenMP (`schedule(dynamic)`), mientras que las funciones `dct::direct` y `dct::inverse` se dejan en serie dentro de cada bloque para evitar paralelismo anidado.

### Tiempos secuenciales

La Tabla 1 recoge los tiempos al ejecutar el programa con `OMP_NUM_THREADS=1` sobre la imagen `face_swap_enhanced.png`, promediando cinco ejecuciones. Aunque la versión está compilada con OpenMP, al limitar a un solo hilo conseguimos una baseline equivalente al código secuencial original. Las dos variantes de DCT dominan claramente el tiempo total: suman aproximadamente el 79 %. Las dos variantes de SRM otro 12 %, y el ELA apenas el 3 %. El bucle sobre bloques de la DCT y la convolución del SRM son, por tanto, los puntos donde la paralelización va a notarse más.

**Cuadro 1.** Tiempos secuenciales por proceso (imagen `face_swap_enhanced.png`, 680×680 píxeles, un único hilo, promedio de 5 ejecuciones).

| Proceso      | Tiempo (ms) |
|--------------|-------------|
| SRM 3x3      | 140         |
| SRM 5x5      | 252         |
| ELA (q=90)   | 100         |
| DCT inversa  | 1668        |
| DCT directa  | 1018        |
| <b>Total</b> | <b>3402</b> |

### Tarea 1.3: Paralelización

#### Paralelización con OpenMP (paralelismo de datos)

La estrategia que hemos seguido combina paralelismo a nivel de operación de imagen con paralelismo a nivel de bloque DCT. Las decisiones más relevantes las contamos a continuación.

**Operadores y transformaciones de `Image<T>`.** Todas las funciones que recorren los píxeles de la imagen (`operator+`, `operator*`, `abs`, `convert<S>`, `to_grayscale`, `normalized` y `convolution`) llevan ahora una directiva `#pragma omp parallel for collapse(2)` sobre los dos bucles externos (filas y columnas). `collapse(2)` es importante porque une los dos bucles en un único espacio de iteración, lo que mejora el balanceo cuando el número de hilos no es divisor exacto del número de filas. Cada iteración escribe en una posición distinta del buffer de salida, así que no hay condiciones de carrera, y todas las variables relevantes se declaran en el cuerpo del bucle (privadas por defecto).

**Convolución.** La función `convolution` de `image.h` es el bucle más caro del análisis SRM. La directiva `#pragma omp parallel for collapse(2)` se aplica solo a los dos bucles externos (filas y columnas de la imagen); el bucle del kernel (3×3 o 5×5) se ejecuta en serie dentro de cada hilo. La razón es que el kernel es muy pequeño y el coste de gestionar paralelismo anidado superaría con creces el trabajo útil que ahorraríamos.

**Normalización con reducción.** `Image::normalized()` y `dct::normalize` necesitan el mínimo y el máximo de la imagen antes de aplicar la transformación. Esto encaja con las cláusulas `reduction(min:min_value)` y `reduction(max:max_value)` de OpenMP 3.1, que crean una copia privada por hilo y combinan los resultados al final. El segundo bucle (la transformación píxel a píxel) es trivialmente paralelo y solo lleva `collapse(2)`.

**Bucle sobre bloques en `compute_dct`.** El bucle externo de `compute_dct` que recorre el vector de bloques se paraleliza con `#pragma omp parallel for schedule(dynamic)`. Cada iteración asigna su propia matriz auxiliar local (`dct::create_matrix`), aplica `dct::direct` sobre ella, opcionalmente reconstruye el bloque con `dct::inverse` y libera la memoria. Hemos elegido `dynamic` porque la rama `if (invert)` introduce un coste adicional en los bloques de la versión inversa (anular bajas frecuencias y aplicar `dct::inverse` entera), y un reparto estático dejaría a unos hilos cargados de bloques caros mientras otros terminan sus bloques baratos.

**Decisión sobre `dct::direct` y `dct::inverse`.** `dct::direct` y `dct::inverse` en `dct.cc` contienen bucles cuádruples sobre cada bloque. Una opción habría sido paralelizar también esos bucles internos, pero hemos optado por dejarlos en serie porque ya están envueltos por el bucle externo de bloques, que sí está paralelizado. Activar paralelismo también ahí provocaría *oversubscription* de hilos: un bloque solo tiene  $8 \times 8 = 64$  posiciones y la sobrecarga de la región paralela superaría al trabajo útil. La directiva queda comentada en el código fuente, junto a un comentario que justifica la decisión y permite probar la configuración alternativa rápidamente si hiciera falta.

**Variable de inducción de `compute_dct`.** Una corrección pequeña pero conviene mencionarla: el bucle original recorría los bloques con `int i; i < blocks.size()`, comparando un `int` con un `size_t`. OpenMP exige que la variable de inducción de un bucle paralelo sea de tipo entero con signo, así que hemos mantenido `int` y hemos añadido el casting `(int)blocks.size()` en la condición.

### Paralelización con `std::async` (paralelismo funcional)

Como los cinco procesos de análisis forense (SRM 3x3, SRM 5x5, ELA, DCT inversa y DCT directa) son completamente independientes y no tienen dependencias de datos (todos leen de la misma imagen y generan salidas distintas), hemos aplicado un modelo de paralelismo de tareas con `std::async`.

En el programa principal (`main.cc`) hemos sustituido las llamadas secuenciales por el lanzamiento de cinco tareas asíncronas con la política `std::launch::async`, que obliga al runtime de C++ a crear un hilo independiente por cada proceso. Un detalle de implementación importante: hemos envuelto el argumento de la imagen original con `std::ref(image)`. Por defecto, `std::async` hace copias por valor de los argumentos para garantizar la seguridad entre hilos. Pero hacer cinco copias profundas de una imagen de alta resolución en RAM hubiera supuesto un coste prohibitivo en memoria y rendimiento. Con `std::ref` forzamos el paso por referencia constante, y al ser operaciones de solo lectura es totalmente seguro.

Una vez lanzadas las tareas, el hilo principal se sincroniza con el método `get()` de cada `std::future`, que bloquea la ejecución hasta que la tarea correspondiente termina y extrae la imagen resultante para guardarla en disco.

**Limitación arquitectónica del proceso ELA:** hay un punto que conviene comentar sobre la función `compute_ela`. Este proceso almacena temporalmente

la imagen en disco (`_temp.jpg`) para aplicar la compresión JPEG. En nuestro diseño no es problema porque el proceso ELA se invoca una sola vez. Pero si en el futuro se escalara la aplicación para analizar lotes de imágenes a la vez lanzando varios ELA en paralelo, se produciría una condición de carrera (*race condition*) a nivel de sistema de archivos: varios hilos intentarían sobrescribir y leer el mismo `_temp.jpg` de forma concurrente y los datos quedarían corruptos. La solución sería generar nombres de archivo temporales únicos (con un UUID o el ID del hilo) para cada ejecución.

### Versión combinada (OpenMP + `std::async`)

La versión combinada tiene una jerarquía de dos niveles: paralelismo de tareas en el nivel exterior con `std::async` (lanzando los cinco procesos a la vez) y paralelismo de datos en el nivel interior con OpenMP (repartiendo los bucles dentro de cada proceso).

El reto principal es el *oversubscription* de hilos. Si lanzamos cinco tareas pesadas a la vez y cada una crea su propio equipo de hilos OpenMP asumiendo que tiene la máquina entera para ella sola (por ejemplo, 16 hilos), el SO tendrá que planificar  $5 \times 16 = 80$  hilos activos sobre una CPU de 16 hilos lógicos. El coste de cambio de contexto degradaría el rendimiento hasta dejarlo por debajo de la versión secuencial.

Para evitarlo es fundamental controlar el número de hilos OpenMP por tarea. Una forma es estática, con `omp_set_num_threads(N)` antes de lanzar las tareas asíncronas, ajustando  $N$  de modo que  $N \times (\text{tareas concurrentes}) \approx \text{hilos lógicos del sistema}$ . En nuestro caso, para una máquina de 16 hilos,  $N = 3$  o  $N = 4$  da un balance bastante bueno: todos los procesos corren a la vez y dentro paralelizan sus bucles internos sin saturar al planificador.

La Tabla 2 recoge los tiempos de la versión combinada para distintos valores de `OMP_NUM_THREADS`. El speed-up se calcula respecto al tiempo secuencial puro (3402 ms con un único hilo OpenMP y sin `async`).

**Cuadro 2.** Tiempos de la versión combinada (`std::async` + OpenMP) según el número de hilos OpenMP por proceso.

| Hilos OMP | Tiempo (ms) | Speed-up | Eficiencia |
|-----------|-------------|----------|------------|
| 1         | 2756        | 1,23     | 1,23       |
| 2         | 1416        | 2,40     | 1,20       |
| 4         | 892         | 3,82     | 0,95       |
| 8         | 712         | 4,78     | 0,60       |
| 16        | 679         | 5,01     | 0,31       |

Un detalle interesante de la Tabla 2: con 1 y 2 hilos OpenMP la eficiencia teórica resulta superior a la unidad. No es un error de medida, sino un efecto directo de combinar dos niveles de paralelismo. Aunque cada proceso disponga de un solo hilo OpenMP, las cinco tareas asíncronas se ejecutan a la vez en

distintos núcleos físicos, así que el speed-up respecto al secuencial puro (que solo usa un núcleo) supera al número nominal de hilos OpenMP. La eficiencia vuelve a bajar de uno cuando el número total de hilos activos ( $5 \times \text{HilosOMP}$ ) supera el número de núcleos físicos disponibles, momento en el que aparece el oversubscription y la contención en cachés.

### Resultados de la versión OpenMP

La Tabla 3 recoge los tiempos totales medios, el speed-up y la eficiencia para distintas configuraciones de hilos. La Tabla 4 desglosa el tiempo de cada proceso individualmente. Las gráficas asociadas (Figuras 5, 6, 7 y 8) visualizan el comportamiento.

**Cuadro 3.** Tiempos de ejecución total, speed-up y eficiencia según el número de hilos (versión OpenMP, promedio de 5 ejecuciones).

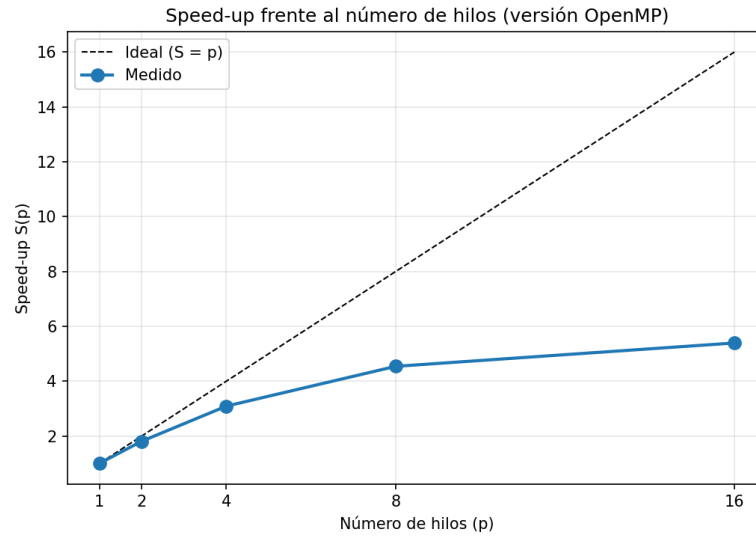
| Hilos | Tiempo (ms) | Speed-up | Eficiencia |
|-------|-------------|----------|------------|
| 1     | 3402        | 1,00     | 1,00       |
| 2     | 1880        | 1,81     | 0,90       |
| 4     | 1101        | 3,09     | 0,77       |
| 8     | 749         | 4,54     | 0,57       |
| 16    | 631         | 5,39     | 0,34       |

**Cuadro 4.** Tiempos medios por proceso (ms) según el número de hilos.

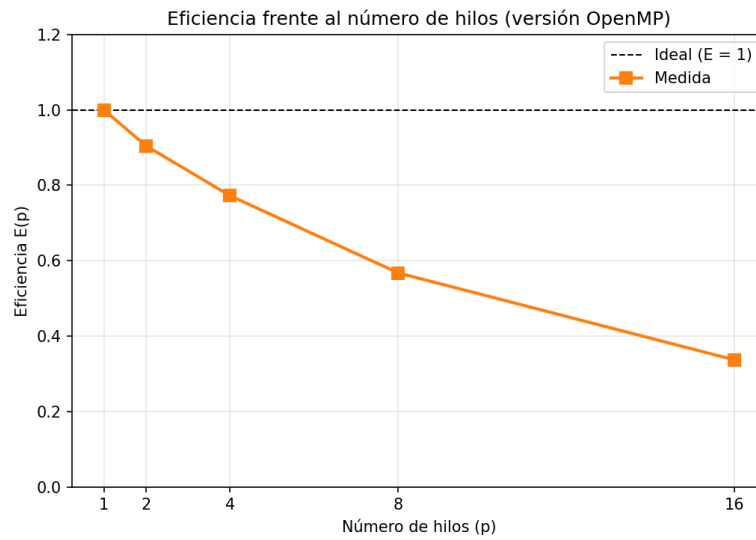
| Hilos | SRM 3x3 | SRM 5x5 | ELA | DCT inv | DCT dir |
|-------|---------|---------|-----|---------|---------|
| 1     | 140     | 252     | 100 | 1668    | 1018    |
| 2     | 70      | 131     | 59  | 865     | 533     |
| 4     | 37      | 75      | 41  | 455     | 271     |
| 8     | 26      | 52      | 33  | 253     | 157     |
| 16    | 23      | 41      | 35  | 184     | 110     |

### Discusión: respuestas a las preguntas planteadas

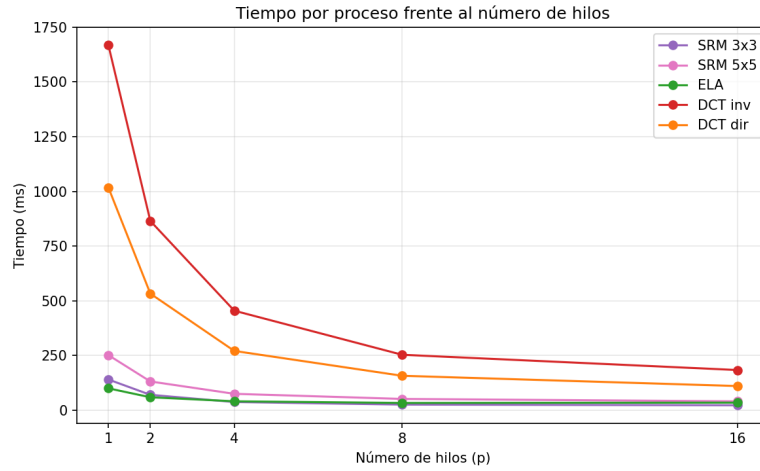
**¿Se obtiene un mejor rendimiento paralelizando todas las partes posibles?** En tiempo total absoluto, sí: cada vez que aumentamos el número de hilos baja el tiempo total, de 3402 ms con un hilo a 631 ms con dieciséis. Sin embargo, la eficiencia decae con rapidez al aumentar el número de hilos (0,90 con dos, 0,77 con cuatro, 0,57 con ocho y 0,34 con dieciséis), lo que indica que la mayor parte de la ganancia se obtiene con los primeros hilos y que añadir más allá de los núcleos físicos disponibles aporta cada vez menos. Además hay procesos cortos como el ELA que apenas se benefician (solo  $2,9\times$  con dieciséis hilos) porque la



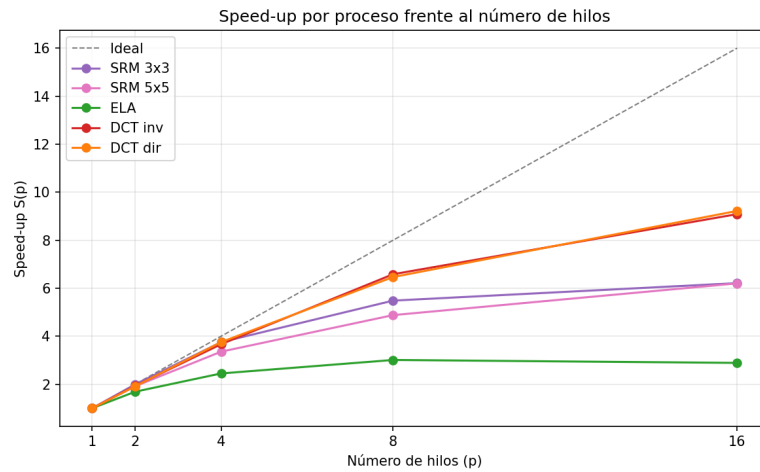
**Figura 5.** Speed-up frente al número de hilos. La línea discontinua representa la curva ideal  $S(p) = p$ .



**Figura 6.** Eficiencia frente al número de hilos. El valor ideal es  $E(p) = 1$ .



**Figura 7.** Tiempo de cada proceso frente al número de hilos.



**Figura 8.** Speed-up individual de cada proceso frente al número de hilos. Las DCT escalan muy bien (hasta 9,2 $\times$ ), mientras que el ELA queda limitado por la I/O serie del JPEG temporal (apenas 2,9 $\times$ ).



parte de I/O del JPEG temporal no es paralelizable y se convierte en cuello de botella, mientras que la DCT —donde hay miles de bloques independientes— llega a un speed-up cercano a  $9,2\times$ . En definitiva, paralelizar todo lo posible mejora el rendimiento global, pero el retorno por hilo añadido no es uniforme y depende mucho del trabajo paralelizable que tenga cada proceso.

**¿Se degrada el rendimiento al paralelizar ciertas partes?** En las pruebas no se ha visto degradación absoluta, pero sí una saturación clara entre 8 y 16 hilos. El speed-up total pasa de  $4,54\times$  a  $5,39\times$  (apenas un 19 % más) a costa de duplicar el número de hilos, y la eficiencia cae casi a la mitad. Algunas funciones cortas, como `to_grayscale` o `convert` sobre la imagen completa, llegan a tener un coste casi imperceptible (pocos milisegundos) en el que la sobrecarga de gestionar la región paralela empieza a ser comparable al trabajo útil. En el ELA el tiempo prácticamente se estanca a partir de 8 hilos e incluso aumenta ligeramente entre 8 y 16 hilos (de 33 ms a 35 ms), un síntoma claro de que la sobrecarga ha superado al beneficio.

**¿A qué se debe esa degradación?** Las causas son varias y se combinan. La primera es el Hyper-Threading: la máquina de pruebas tiene dieciséis hilos lógicos pero ocho núcleos físicos, así que pasar de ocho a dieciséis hilos no añade hardware nuevo, sino que comparte unidades de ejecución entre dos hilos por núcleo, con la contención que eso supone en cachés y unidades funcionales. La segunda es la sobrecarga propia de OpenMP: crear y sincronizar el equipo de hilos en cada `#pragma omp parallel for` tiene un coste fijo que resulta apreciable cuando el bucle es corto (en `to_grayscale` sobre una imagen pequeña, por ejemplo). La tercera es la fracción serie del programa, dominada por la I/O de PNG y JPEG (lectura de la imagen al inicio, escritura de las cinco salidas y el JPEG temporal del ELA): por la Ley de Amdahl, esta parte pone un techo al speed-up alcanzable, da independientemente del número de hilos. Por último, en bucles muy largos sobre toda la imagen, también contribuye la *falsa compartición* (false sharing) cuando varios hilos escriben en líneas de caché contiguas, aunque su impacto en este problema es menor que las anteriores.

**¿Cuál es la mejor estrategia de paralelización?** A la vista de los resultados, la estrategia ganadora es paralelizar a granularidad gruesa allí donde haya muchas unidades de trabajo independientes y dejar en serie todo aquello cuyo coste sea inferior al de gestionar la región paralela. En esta práctica eso se traduce en paralelizar el bucle externo sobre los 7 225 bloques de la DCT (que da  $9\times$  de speed-up), paralelizar la convolución del SRM con `collapse(2)` sobre los dos bucles más externos (que da  $6\times$ ) y dejar las funciones internas de `dct::direct` y `dct::inverse` en serie para evitar paralelismo anidado innecesario. Con la máquina de pruebas, el punto óptimo de configuración está sobre los 8 hilos: con esa configuración se obtiene un speed-up de  $4,54\times$  con una eficiencia razonable de 0,57, mientras que pasar a 16 hilos solo añade un 19 % adicional de speed-up a cambio de reducir la eficiencia hasta 0,34.

## Tarea 2: Memoria final

### Sobre el código implementado

**Talla del problema.** La talla del problema viene dada por las dimensiones de la imagen de entrada (anchura, altura y número de canales) y por varios parámetros de los procesos de análisis: el tamaño del kernel SRM (3 o 5), el tamaño de bloque DCT (8 por defecto) y el factor de calidad JPEG en el ELA (90 en la configuración actual). Para la imagen `face_swap_enhanced.png`, de  $680 \times 680$  píxeles RGB, esto se traduce en  $\sim 1,4$  millones de píxeles a procesar por convolución y 7 225 bloques de  $8 \times 8$  a procesar por DCT.

**Estructuras de control de especial interés.** Hay tres estructuras especialmente relevantes desde el punto de vista del paralelismo. La primera es el bucle cuádruple de `Image::convolution`, que recorre filas y columnas de la imagen y, dentro, las dos dimensiones del kernel: es el bucle más caro del análisis SRM y se paraleliza con `collapse(2)` sobre los dos bucles externos. La segunda es el bucle cuádruple de `dct::direct` y `dct::inverse`, que se ejecuta una vez por bloque  $8 \times 8$  y permanece en serie para no provocar paralelismo anidado. La tercera es el bucle externo sobre el `std::vector<Block>` en `compute_dct`, que es el que sí se paraleliza con OpenMP usando `schedule(dynamic)` para repartir bien la carga entre las dos variantes de DCT.

**Cambios respecto a la versión secuencial.** Respecto a la versión secuencial, los cambios principales son: incluir la cabecera `<omp.h>` en `main.cc`, `image.h` y `dct.cc`, e introducir las directivas `#pragma omp parallel for` en los puntos descritos en la Tarea 1.3. En `dct.cc` hemos movido la declaración de las variables auxiliares `ci`, `cj`, `Cu` y `Cv` al cuerpo del bucle para que sean privadas por iteración. En `compute_dct` hemos ajustado el tipo de la variable de inducción y la condición del bucle para cumplir con los requisitos de OpenMP. Por último, al inicio de `main.cc` hemos añadido una pequeña región paralela que imprime el número de hilos activos, lo cual resulta útil para verificar que `OMP_NUM_THREADS` ha hecho efecto al ejecutar.

A nivel funcional, el cambio más importante respecto a la versión secuencial es la sustitución de las cinco llamadas en serie a los procesos de análisis por cinco lanzamientos asíncronos con `std::async` usando `std::ref(image)` para evitar copias profundas de la imagen entre hilos, y la sincronización posterior con `std::future::get()` para recoger los resultados antes de guardarlos en disco. Esto exige incluir la cabecera `<future>` y reorganizar el flujo de control de `main` para que las cinco tareas se lancen antes de empezar a guardar resultados.

**Instrucciones OpenMP y `std::async` empleadas.** Las directivas y funciones de OpenMP que hemos usado son: `#pragma omp parallel for collapse(2)` en los operadores y transformaciones de `Image<T>` y en la convolución; `#pragma omp parallel for collapse(2) reduction(min:...)` `reduction(max:...)` en

el primer bucle de `Image::normalized` y `dct::normalize` para calcular el mínimo y el máximo de la imagen sin condiciones de carrera; `#pragma omp parallel for schedule(dynamic)` en el bucle externo de `compute_dct` sobre el vector de bloques; y `omp_get_num_threads()` dentro de una región paralela con `#pragma omp single` para mostrar el número de hilos al inicio de la ejecución.

Por la parte de programación asíncrona, hemos usado `std::async(std::launch::async, ...)` para lanzar las cinco tareas independientes, `std::ref(image)` para pasar la imagen original por referencia constante a cada tarea sin copia profunda, `std::future<Image<unsigned char>` como tipo de retorno para cada tarea y el método `get()` sobre cada future para sincronizar el hilo principal con el final de cada tarea y recuperar la imagen resultante.

### Sobre el paralelismo explotado

**Tipos de paralelismo usados.** La aplicación combina dos tipos de paralelismo. Uno es paralelismo de datos (data-parallel) implementado con OpenMP en los bucles que recorren los píxeles de la imagen y los bloques DCT, donde la misma operación se aplica de forma independiente a distintas porciones de los datos. El otro es paralelismo funcional (task-parallel) implementado con `std::async` sobre los cinco procesos independientes (SRM 3x3, SRM 5x5, ELA, DCT inversa y DCT directa), donde cada hilo ejecuta una tarea distinta sobre la misma imagen.

**Modo de programación paralela.** El modo es de variables compartidas en una máquina paralela de memoria centralizada (SMP, Symmetric Multi-Processing). Todos los hilos del programa ven el mismo espacio de memoria y comparten estructuras como la imagen de entrada o el vector de bloques DCT sin necesidad de pasar mensajes explícitos.

**Alternativas de comunicación.** La comunicación entre hilos se hace de forma implícita a través de la memoria compartida: cuando un hilo escribe en una posición del buffer de salida, el resultado queda visible para los demás sin necesidad de operaciones explícitas de envío o recepción. La única comunicación explícita en el programa es la sincronización con `std::future::get()` cuando se recogen los resultados de las cinco tareas lanzadas con `std::async`.

**Estilo de programación paralela.** El estilo en OpenMP es SPMD (Single Program Multiple Data) con modelo fork-join: en cada `#pragma omp parallel for` el hilo maestro hace *fork* de un equipo de hilos que ejecutan el mismo bucle sobre datos distintos y, al terminar, hacen *join* de vuelta al hilo maestro. Para `std::async` el estilo es paralelismo basado en tareas, donde cada tarea es una función completa que se sincroniza con el hilo principal mediante `std::future`.

**Tipo de estructura paralela.** La estructura final es híbrida y de dos niveles. En el exterior, las cinco tareas independientes se ejecutan en paralelo con `std::async`. En el interior, los bucles sobre los datos dentro de cada tarea se paralelizan con OpenMP. Combinar ambos niveles obliga a controlar el número total de hilos para evitar oversubscription, ya que si no podríamos crear cinco veces más hilos OpenMP que núcleos físicos disponibles.

### Caracterización de la máquina paralela

Las pruebas se han hecho en el entorno de desarrollo de la Tabla 5, una máquina con procesador AMD Ryzen 7 7435HS de 8 núcleos físicos y 16 hilos lógicos accesible desde Ubuntu sobre WSL2. Las características relevantes para interpretar los resultados son la presencia de SMT (dos hilos por núcleo físico) y la jerarquía de cachés, con una caché L3 de 16 MiB compartida por todos los núcleos.

**Cuadro 5.** Características de la máquina paralela utilizada para las medidas (entorno de desarrollo, Ubuntu sobre WSL2).

| Componente            | Valor                              |
|-----------------------|------------------------------------|
| CPU                   | AMD Ryzen 7 7435HS                 |
| Arquitectura          | x86_64                             |
| Núcleos físicos       | 8                                  |
| Hilos lógicos         | 16 (2 por núcleo, SMT)             |
| SMT / Hyper-threading | Sí (flag <code>ht</code> presente) |
| Caché L1d             | 256 KiB (32 KiB por núcleo)        |
| Caché L1i             | 256 KiB (32 KiB por núcleo)        |
| Caché L2              | 4 MiB (512 KiB por núcleo)         |
| Caché L3              | 16 MiB (compartida)                |
| RAM (asignada a WSL)  | 11 GiB                             |
| SO                    | Ubuntu sobre WSL2                  |

**¿Qué significa la palabra `ht`? ¿Aparece en nuestro ordenador de prácticas?** La flag `ht` en la salida de `cat /proc/cpuinfo` es la abreviatura de *Hyper-Threading*, una tecnología desarrollada originalmente por Intel que permite que cada núcleo físico ejecute dos hilos lógicos compartiendo la mayoría de las unidades de ejecución pero manteniendo registros y estado independientes. AMD tiene una tecnología equivalente con otro nombre, *Simultaneous Multi-Threading* (SMT), pero por compatibilidad histórica el campo en `/proc/cpuinfo` se sigue llamando `ht` en los dos fabricantes.

En la máquina que hemos usado para las pruebas (AMD Ryzen 7 7435HS) la flag `ht` aparece, lo que confirma que el SMT está activo. Se confirma también con el campo `Thread(s) per core: 2` de `lscpu` y con la presencia de dieciséis hilos lógicos sobre ocho núcleos físicos. Este dato es clave para interpretar los

resultados experimentales: el speed-up casi lineal hasta ocho hilos refleja que estamos usando bien los ocho núcleos físicos, mientras que la fuerte caída de eficiencia entre ocho y dieciséis hilos se debe a que a partir de ahí los hilos compiten por los recursos compartidos dentro de cada núcleo.

## Resultados experimentales

**Tabla de tiempos por número de hilos.** Los resultados detallados ya están en la Tarea 1.3 (Tablas 3 y 4 y Figuras 5 a 8). En resumen, el tiempo total pasa de 3402 ms con un hilo a 631 ms con dieciséis, lo que supone un speed-up global de  $5,39\times$  y una eficiencia de 0,34 en esa configuración. Con 8 hilos se obtiene un compromiso mejor:  $4,54\times$  de speed-up y 0,57 de eficiencia.

**Speed-up máximo teórico.** Aplicando la Ley de Amdahl, el speed-up máximo se calcula como  $S_{max}(p) = 1/(f + (1-f)/p)$ , donde  $f$  es la fracción serie del programa. La parte estrictamente no paralelizable son la lectura de la imagen de entrada, la escritura de los cinco PNG de salida y la I/O del JPEG temporal del ELA. Mirando el código y los tiempos medidos, estimamos  $f \approx 0,03$  (aproximadamente el 3 % del tiempo total). Con ese valor, el speed-up máximo teórico para infinitos hilos sería  $S_{max}(\infty) = 1/0,03 \approx 33$ . Para 16 hilos, la cota teórica es  $S_{max}(16) = 1/(0,03 + 0,97/16) \approx 10,9$ , valor por encima del speed-up real de  $5,39$  obtenido. La diferencia se explica por el efecto del Hyper-Threading, la sobrecarga propia de OpenMP y la contención de caché entre hilos.

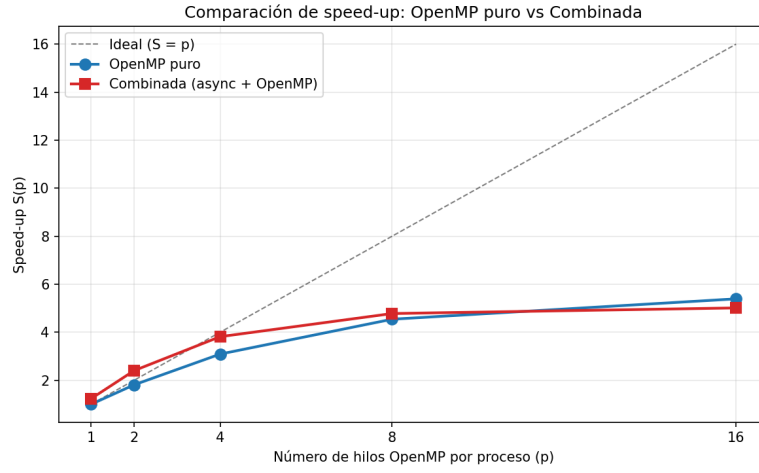
**Comentario razonado de los resultados.** El comportamiento observado se ajusta al patrón típico de un programa paralelo bien diseñado pero limitado por su fracción serie y por el hardware. Hasta cuatro hilos el speed-up es casi proporcional al número de hilos (eficiencias entre 0,77 y 0,90), porque cada hilo se asigna a un núcleo físico distinto y dispone de sus propias unidades de ejecución. Al pasar a ocho hilos la eficiencia ya cae a 0,57: empezamos a saturar los ocho núcleos físicos y la sobrecarga de sincronización en bucles cortos resulta perceptible. De ocho a dieciséis hilos la mejora ya es marginal porque cada hilo físico tiene que compartir su núcleo con otro hilo lógico, y los recursos compartidos (caché L1/L2, unidades de cálculo) limitan el rendimiento adicional. El proceso ELA es el ejemplo más claro del techo que pone la fracción serie: solo escala  $2,9\times$  con dieciséis hilos porque la mayor parte de su tiempo se va en escribir y leer el JPEG temporal, una operación que no se beneficia del paralelismo.

## Comparación final entre las dos implementaciones

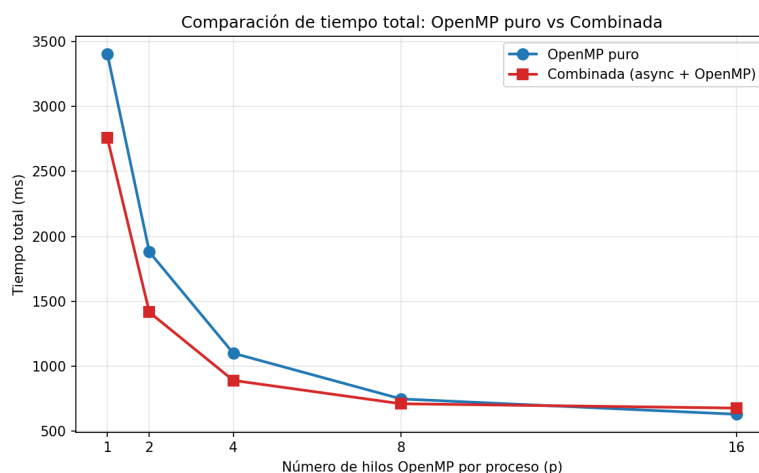
La Tabla 6 y la Figura 9 resumen el comportamiento de las dos implementaciones (OpenMP puro y combinada `std::async` + OpenMP) para distintos valores de `OMP_NUM_THREADS`. La línea base de speed-up es el tiempo secuencial puro de 3402 ms (un único hilo OpenMP, sin `async`).

**Cuadro 6.** Comparación de tiempos totales entre la versión OpenMP puro y la versión combinada para distintos valores de `OMP_NUM_THREADS`.

| Hilos OMP | OpenMP puro (ms) | Combinada (ms) | Mejora  |
|-----------|------------------|----------------|---------|
| 1         | 3402             | 2756           | +19,0 % |
| 2         | 1880             | 1416           | +24,7 % |
| 4         | 1101             | 892            | +19,0 % |
| 8         | 749              | 712            | +5,0 %  |
| 16        | 631              | 679            | -7,5 %  |



**Figura 9.** Comparación de speed-up de las dos implementaciones frente al número de hilos OpenMP por proceso. La línea discontinua representa la curva ideal  $S(p) = p$ .



**Figura 10.** Comparación de tiempo total absoluto de las dos implementaciones frente al número de hilos OpenMP por proceso.

A la vista de los resultados, la versión combinada gana a la OpenMP puro en todas las configuraciones excepto en la de 16 hilos. La mejora es más significativa con valores bajos o intermedios de `OMP_NUM_THREADS`: con un solo hilo OpenMP la combinada va un 19 % más rápida, y con dos hilos la mejora llega al 24,7 %. La explicación es directa: cuando OpenMP solo tiene uno o dos hilos por proceso no puede explotar la mayoría de los núcleos físicos del sistema, mientras que la versión combinada lanza las cinco tareas asíncronas a la vez y ocupa la máquina aprovechando hasta cinco núcleos simultáneamente.

A medida que crece el número de hilos OpenMP la diferencia se estrecha porque OpenMP ya es capaz por sí solo de ocupar buena parte del hardware. Con 8 hilos la diferencia es del 5 %, y con 16 hilos la combinada termina siendo más lenta. La razón es el oversubscription: con la combinada hay  $5 \times 16 = 80$  hilos OpenMP activos compitiendo por 16 hilos lógicos, lo que multiplica los cambios de contexto y la presión sobre la caché L3 compartida. La OpenMP pura, en cambio, mantiene exactamente 16 hilos activos en todo momento.

La configuración óptima global es la **versión combinada con 8 hilos OpenMP**, que da  $4,78\times$  de speed-up con un tiempo total de 712 ms y una eficiencia razonable de 0,60. Esta configuración aprovecha bien los 8 núcleos físicos de la máquina sin saturar el SMT y, gracias al `async`, mantiene siempre los cinco procesos ocupando el hardware en paralelo. Como recomendación práctica, en sistemas con más núcleos físicos la configuración óptima de `OMP_NUM_THREADS` para la combinada se aproxima al cociente entre el número de hilos lógicos del sistema y el número de tareas asíncronas lanzadas (en este caso,  $16/5 \approx 3$ , aunque empíricamente los valores entre 4 y 8 funcionan mejor por la fracción serie de algunos procesos como el ELA).

En conjunto, los resultados confirman lo que esperábamos: el paralelismo de tareas con `std::async` y el paralelismo de datos con OpenMP se complementan, y combinarlos da mejor rendimiento que aplicar cualquiera de los dos por separado, siempre que se controle el número total de hilos para no caer en oversubscription.

## Disponibilidad del código

Todo el código fuente desarrollado en esta práctica (versión OpenMP, versión combinada con `std::async`, scripts de benchmark y diagramas) está disponible públicamente en el siguiente repositorio de GitHub:

[https://github.com/Ximsig/Practica\\_4](https://github.com/Ximsig/Practica_4)

El repositorio incluye los ficheros fuente comentados (`main.cc`, `image.h`, `dct.cc`), los scripts para automatizar las medidas (`bench.sh` y `plot_results.py`) y los datos crudos de los benchmarks en formato CSV, lo cual permite reproducir todos los resultados que aquí presentamos.

## Referencias

1. OpenMP Architecture Review Board: OpenMP API specification. <https://www.openmp.org>, último acceso 2026.
2. cppreference.com: `std::async`. <https://en.cppreference.com/w/cpp/thread/async>, último acceso 2026.
3. Lawrence Livermore National Laboratory: OpenMP Tutorial. <https://computing.llnl.gov/tutorials/openMP/>, último acceso 2026.